Base de la documentation technique - Jour 2

Sommaire

Base de la documentation technique - Jour 2..... 1
Sommaire..... 2
Méthodologie de projet..... 3
1. Description de la méthodologie.....3
2. Pourquoi cette méthodologie est adaptée au projet..... 3
Environnement de développement.....5
Choix techniques - Meadowbound..... 6
1. Langage de programmation : C#.....6
2. Framework de jeu : MonoGame..... 6
3. Gestion des versions : Git et GitHub.....6
4. Stockage des données : JSON envisagé..... 6
5. Organisation du code.....6
Sources.....7
Fonctionnalités attendues dans le MVP.....7
Critère global de validation du MVP.....8
Utilisation de l'ia.....10

Méthodologie de projet

1. Description de la méthodologie

La méthodologie retenue pour le développement du jeu repose sur la combinaison de deux approches : Component-Driven Truncated Agile (CDTA) et Architecture-First Vertical Slice.

Le Component-Driven Truncated Agile consiste à organiser le développement autour de composants distincts, en adoptant un processus Agile simplifié et adapté aux contraintes du projet. Le développement est réalisé progressivement, avec des cycles de travail courts, des objectifs limités et des vérifications régulières.

L'approche Architecture-First Vertical Slice consiste à définir en premier lieu les bases de l'architecture technique, puis à développer des portions fonctionnelles du jeu intégrant plusieurs systèmes. Chaque vertical slice permet de vérifier qu'un ensemble de fonctionnalités fonctionne correctement dans le jeu avant de poursuivre son développement.

La combinaison de ces deux approches vise à maintenir une architecture cohérente tout en permettant un développement progressif, flexible et centré sur l'obtention de versions fonctionnelles.

2. Pourquoi cette méthodologie est adaptée au projet

Cette combinaison est adaptée au développement d'un jeu de farming en 2.5D avec C# et MonoGame, car le projet comporte plusieurs systèmes interdépendants, tels que les déplacements du joueur, la gestion de la carte, les collisions, l'agriculture, l'inventaire et les sauvegardes.

L'organisation par composants permet de limiter le couplage entre ces systèmes et de faciliter leur maintenance. L'approche Architecture-First permet de réfléchir à leur organisation avant que le projet ne devienne trop complexe. Enfin, les vertical slices permettent de vérifier régulièrement que les différents composants fonctionnent correctement ensemble.

Cette méthodologie est également adaptée au fait que je suis seul responsable de l'ensemble du projet. J'assure notamment les rôles de chef de projet, développeur, testeur et opérateur. Un processus Agile simplifié me permet donc de limiter les tâches administratives, de concentrer mon temps sur le développement et les tests, et d'adapter les priorités en fonction des difficultés rencontrées.

Le projet progressera ainsi par étapes, avec pour objectif de disposer d'une version fonctionnelle à la fin de chaque cycle de travail hebdomadaire, tout en préservant les

fonctionnalités déjà implémentées. La durée totale et le contenu des différentes étapes resteront adaptables en fonction de l'avancement réel du projet.

Environnement de développement

Le projet Meadowbound est un jeu de farming en 2.5D développé en C# à l'aide du framework MonoGame. Le développement sera réalisé dans un environnement permettant d'écrire, d'exécuter, de déboguer et de tester le code du jeu.

Git sera utilisé pour le suivi des versions du code source, tandis que GitHub servira à héberger le dépôt du projet et à organiser les tâches de développement.

L'environnement sera organisé de manière à faciliter le développement progressif des composants, les tests réguliers et la maintenance du projet.

Bien sûr ! On va rédiger la partie Choix techniques de Meadowbound, dans le même style que la méthodologie : un texte clair, professionnel et suffisamment concis pour être intégré directement à ton document.

L'objectif est de justifier les outils retenus, sans figer trop tôt les détails techniques qui pourront évoluer pendant le développement.

Choix techniques - Meadowbound

1. Langage de programmation : C#

Le langage C# a été choisi pour développer Meadowbound, car il permet de structurer le code à l'aide de classes et de composants distincts. Il convient à la création de systèmes de jeu tels que les déplacements, l'agriculture, l'inventaire et la gestion des sauvegardes. Il s'intègre également directement à MonoGame.

2. Framework de jeu : MonoGame

MonoGame a été retenu comme framework de développement. Il fournit les outils nécessaires pour gérer le rendu graphique, les entrées du joueur, le son et les ressources du jeu. Contrairement à un moteur de jeu complet, il laisse davantage de liberté dans la conception de l'architecture et dans l'implémentation des systèmes. Ce fonctionnement correspond à l'approche Architecture-First choisie pour le projet.

3. Gestion des versions : Git et GitHub

Git sera utilisé pour suivre les modifications du code source et conserver un historique des versions. GitHub hébergera le dépôt du projet et permettra d'organiser les tâches à l'aide d'issues.

Ces outils faciliteront le suivi de l'avancement, la correction des erreurs et le retour à une version antérieure si nécessaire. Ils seront également utiles pour maintenir une organisation claire, même dans le cadre d'un développement individuel.

4. Stockage des données : JSON envisagé

Le format JSON est envisagé pour stocker certaines données du jeu, notamment les objets, les cultures et les paramètres de configuration. Il pourrait également être utilisé pour les sauvegardes si la structure des données s'y prête.

Ce format lisible facilite l'inspection et la modification des données pendant le développement. Le choix définitif de la structure des sauvegardes sera effectué lorsque les besoins du jeu seront mieux définis.

5. Organisation du code

Le code sera organisé en composants et en systèmes ayant chacun une responsabilité précise. Cette organisation vise à limiter les dépendances entre les fonctionnalités et à faciliter les tests, la maintenance et les évolutions futures.

La structure exacte du projet sera définie progressivement, en fonction des besoins identifiés lors du développement des premières vertical slices.

Sources

- MonoGame — Documentation officielle.
- Microsoft — Présentation du langage C#.

Fonctionnalités attendues dans le MVP

Le MVP (Minimum Viable Product) de Meadowbound correspond à une première version jouable qui intègre les mécaniques essentielles du jeu. L'objectif est de valider le fonctionnement des systèmes principaux avant d'ajouter des fonctionnalités secondaires.

Les fonctionnalités sont classées selon deux niveaux de priorité :

- P0 — Indispensable : nécessaire pour valider le MVP.
- P1 — Secondaire : peut être reporté à une version ultérieure si le temps ou les contraintes techniques l'exigent.

Fonctionnalité	**Priorité**	**Critère de validation**
Affichage de la carte	P0	La carte est chargée et visible sans erreur.
Déplacement du joueur	P0	Le personnage se déplace dans les quatre directions prévues.
Caméra 2.5D	P0	La caméra suit le joueur sans dépasser les limites de la carte.
Collisions	P0	Les obstacles empêchent le passage du personnage.
Tri de profondeur	P0	Le joueur apparaît correctement devant ou derrière les objets selon sa position.
Préparation du sol	P0	Une parcelle valide peut changer d'état.

Fonctionnalité	Priorité	Critère de validation
Plantation	P0	Une graine peut être plantée sur une parcelle préparée et valide.
Croissance	P0	Une plante évolue entre plusieurs stades au fil du temps de jeu.
Récolte	P0	Une plante arrivée à maturité produit une ressource récoltable.
Inventaire	P0	Les ressources récoltées sont stockées et leur quantité est mise à jour.
Sauvegarde et chargement	P0	La progression essentielle est conservée après la fermeture et restaurée au redémarrage.
Interface utilisateur	P0	Le joueur peut comprendre les commandes principales et consulter son inventaire.
Sons et animations avancées	P1	Des effets sonores et des animations supplémentaires peuvent être ajoutés ultérieurement.
Commerce et PNJ	P1	Les interactions commerciales et les personnages non joueurs pourront être intégrés dans une version ultérieure.

Critère global de validation du MVP

Le MVP sera considéré comme validé lorsque le joueur pourra effectuer une boucle de jeu complète :

Se déplacer → préparer une parcelle → planter → attendre la croissance → récolter → sauvegarder → reprendre sa partie.

Cette boucle devra fonctionner sans erreur bloquante et préserver les données essentielles du joueur après le chargement de la sauvegarde.

Les fonctionnalités P0 constituent le périmètre initial de référence. Elles pourront être ajustées si des difficultés techniques importantes apparaissent, à condition de documenter les changements et de préserver l'objectif d'obtenir une version jouable.

Utilisation de l’ia

Durant la rédaction de ce fichier, j’ai fait appelle a chatgpt pour corriger et reformuler de façon professionnelle pour avoir un document présentable et non un document brouillon.